Application Protocol:

Protocol Name: PUI (Pickle Unit Info)

Data Format:

Header: 4 unsigned bytes (length of the message)

Body: Serialize object, that contains a python class

End: '\r\n' indicating the end of the message

Serialization: Pickle module (pickle.dumps( ))

Python Class:

```
# Pickle unit info
class PUI():
    def __init__(self):
        self.action = None  # String request / status code response
        self.path = None   # String file path
        self.recursion = False   # Boolean value for [-R] recursive flag
        self.response = None     # File, Dir or File path send as byte object

    # Serializes the class and makes it ready to send over the socket
    def encapsulate(self):
        serialize_message = pickle.dumps(self)
        message_length = (len(serialize_message) + 2).to_bytes(length=4,byteorder='big')
        return message_length + serialize_message + '\r\n'.encode()
```

Message Flow:

- Client send the request to the server
- If the server receives a valid request and process that request correctly, it will populates the action variable with status code and the response variable with the response (if the client expects a result otherwise the result will be populated with the boolean value of True) and sends it back to the client for use

- If the server receives an invalid request it will populate the action variable with a status error code, and the response variable with an error message, then it sends it back to the client for proper handling.

Behavior:

The header is 4 unsigned bytes that only and only contains the length of the message, all the essential information such as the action, and arguments are inside the serialized object (body of the message).

Serialize object struct data:

Action : string that can only contain Status codes already defined

Path : string use to send the path of request, only used when the client sends the request otherwise is NULL

Recursion : Boolean value indicating the recursion flag, is only put into account if the action is GET or PUT

Response : It can only contain strings, lists, bytes, or booleans depending on the case, is only used by the server otherwise is NULL, if the action is a Status code of error the response will contain an error message.

Based on the action that the client send the server can respond with certain Status codes

- The client is strict to sending only requests status code messages, all other status codes are exclusively reserved for the server and cannot be used by the client.

- After sending the response to a request, the server will be waiting for an OK status code response from the client, if it is not received the server will send an OK? status code message to the server, if no response server will close the connection.

Status Codes:

cd command:

`CD` = Request to change directory to specific path.

`DIR_CHANGED` = Directory changed successfully.

`DIR_NOT_FOUND` = Directory doesn't exist.

ls command:

`LS` = Request to list the contents of a directory.

`OK_LS` = List of the contents of a directory sent.

`DIR_NOT_FOUND` = Directory doesn't exist.

mkdir command:

`MAKE_DIR` = Request to create a directory.

`DIR_CREATED` = Directory created.

`DIR_EXIST` = Directory already exists.

get command:

`GET` = Request to retrieve a path.

`OK_GET` = Path retrieved correctly.

`NO_PATH` = Path was never specified.

`FILE_NOT_FOUND` = File doesn’t exist.

`DIR_NOT_FOUND` = Directory doesn't exist.

put command:

`PUT` = Request to upload and store a path.

`OK_PUT` = Path uploaded and stored correctly on the remote machine

`NO_PATH` = Path was never specified

`FILE_EXIST` = File already exists.